Nav2 - concepts et mise en place

Novembre 2026



Clément Pène - Ingénieur de recherche au LAAS

SOMMAIRE



- Présentation de Nav2
- Comprendre les transformées
- Localisation et fusion de données
- Capteurs disponibles
- SLAM - construire une carte
- Navigation autonome



Présentation de Nav2

Applications

https://docs.nav2.org/jazzy/

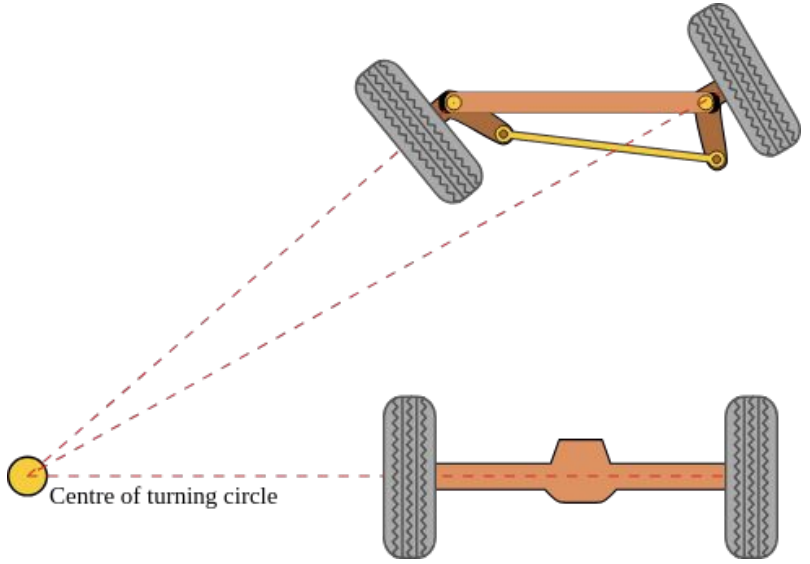
- **Systèmes d'automatisation d'entrepôt**
- **Assistance robotique en santé**
- **Robotique agricole**
- **Services d'accueil/hôtellerie**

Types de robot compatibles

- **Robots à entraînement différentiel**
- **Plateformes holonomes**
- **Systèmes à direction Ackermann**









Comprendre les transformées

Pourquoi des transformées dans nav2 ?

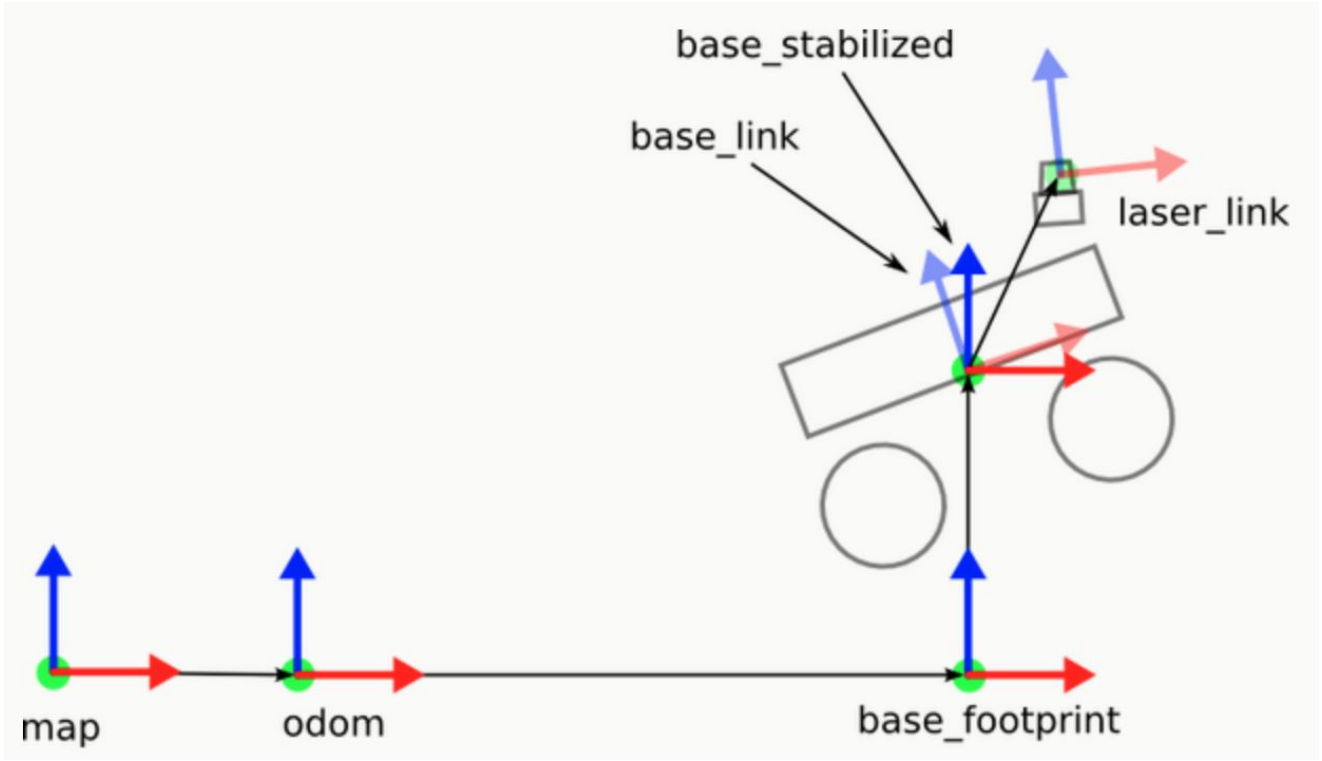
- **Nav2 doit savoir en permanence : où est le robot, où sont les obstacles, où est l'objectif.**
- **Ces informations proviennent de sources différentes -> il faut un langage commun pour les relier**

Les REPs : les règles du jeu

- **REP = ROS Enhancement Proposal, des conventions partagées par tout l'écosystème ROS**
- **REP 105 : Comment nommer et organiser les repères (map, odom, base_link...)**
- **REP 103 : Unités standard (mètres) et conventions d'axes/orientation**
- **But : que n’importe quel paquet Nav2 fonctionne avec n’importe quel robot**

4 repères clés

- **base_link** : centre du robot, bouge avec lui, référence pour tout le reste
- **base_footprint** : projection du robot au sol (contact avec le sol)
- **odom** : point de départ, mesuré par les roues — comme un podomètre (dérive avec le temps)
- **map** : position absolue sur la carte — comme un GPS (précis à long terme, mais peut “sauter” en se recalant)



Les transformées à fournir à Nav2

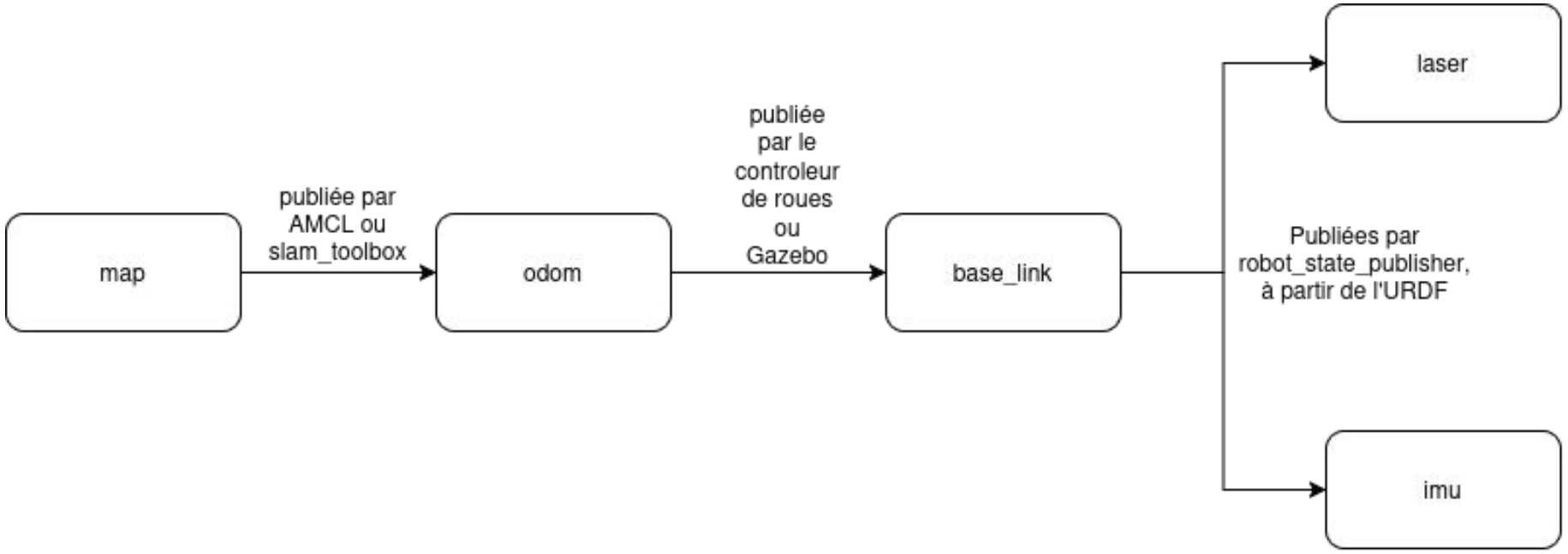
- **map → odom :** publiée par la localisation (AMCL ou SLAM)
- **odom → base_link :** publiée par l'odométrie (roues + IMU)
- **base_link → capteurs :** une par capteur (lidar, caméra...)

STATIC ou DYNAMIQUE ?

- **Static :** Définies dans l'URDF, publiées par robot_state_publisher
- **Dynamique :** Publiées par les paquets de localisation / odométrie

A retenir

- **Localisation :** combiner capteurs + transformées → connaître la pose du robot
- **Planification :** calculer le chemin dans map, puis le traduire dans base_link pour l'exécuter
- **Comportements de secours :** se resituer en cas de problème pendant la navigation



localisation et fusion de données



Pourquoi fusionner les capteurs ?

- corriger le glissement des roues
- navigation plus efficace (moins d’erreurs, trajectoires plus précises)
- Fiabilité : tient le coup même si un capteur tombe en panne temporairement
- Meilleures performances globales

Le filtre de Kalman étendu (EKF)

- Prédit le mouvement attendu, puis corrige avec les mesures réelles des capteurs
- Fait confiance davantage aux capteurs les plus fiables
- Continue de fonctionner même si un capteur s’arrête momentanément
- Filtre de kalman standard -> modèle linéaire. Or ici, orientation du robot non linéaire

Configurer le noeud ekf_filter_node

configuration de robot_localization

documentation officiel Nav2

- **Entrées : /mecanum_drive_controller:odom (roues) + /imu/data (IMU)**
- **Sorties : /odometry/filtered (odométrie fusionnée) + TF odom -> base_footprint**

Paramètres clés d'ekf.yaml :

- **frequency : fréquence de sortie du filtre**
- **two_d_mode : robot sur plan**
- **Frames (REP 105) : map_frame, odom_frame, base_link_frame, world_frame**
- **odom0_config/imu0_config : quels axes de chaque capteur sont utilisés**

Capteurs disponibles

Capteurs et messages standards

- Capteurs courants en robotique mobile :
  - LIDAR
  - IMU
  - caméra (profondeur / RGB)
  - GPS (en exterieur)
- sensor_msgs
  - sensor_msgs/LaserScan: LIDARs
  - sensor_msgs/PointCloud2: caméras profondeur
  - sensor_msgs/Range: capteur de distance (infrarouge / ultrasons)
  - sensor_msgs/Image: caméras

Autres messages standards utiles

- radar_msgs
- vision_msgs



SLAM - construire une carte

Problème de localisation

Pour se localiser, il faut une carte. Pour construire une carte, il faut se localiser -> **SLAM (Simultaneous Localization And Mapping)**

Principe :

- Compare le nouveau scan aux scans précédents / à la carte déjà construite -> estime de combien il a bougé
- Met à jour sa position avec cette estimation
- Ajoute les nouveaux points détectés à la carte, à la position maintenant connue.

La carte est une grille d'occupation : chaque cellule a une probabilité d'être occupée (0 = libre, 1 = obstacle, 0.,5 = inconnu).

On peut voir cette construction via le topic **/map (nav_msgs/OccupancyGrid)**

Configurer slam_toolbox

- l’EKF corrige **odom** en continu avec les capteurs proprioceptifs (roues, IMU).
- SLAM corrige **map** par rapport à **odom** avec les capteurs extéroceptifs (LIDAR).

Voir l’étape du readme pour configurer slam_toolbox et lancer la génération de la carte.

Navigation autonome

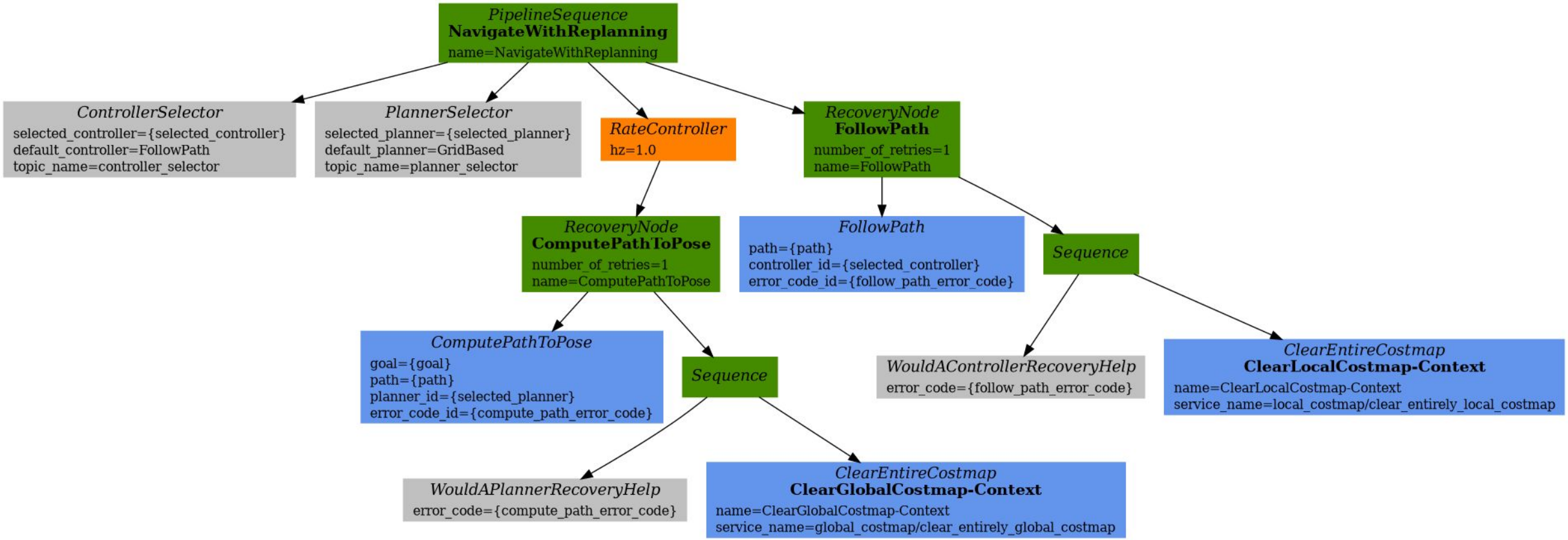
Configuration

Suivre le README pour configurer les éléments de navigation :

- amcl
- global_costmap
- local_costmap
- bt_navigator
- controller_server
- velocity_smoother
- planner_server
- smoother_server
- behavior_server
- collision_monitor

Behavior Tree : orchestrateur

[arbre de navigation par défaut]



Visualiser les arbres disponibles

- ls $(ros2 pkg prefix nav2_bt_navigator)/share/nav2_bt_navigator/behavior_trees/
- cat $(ros2 pkg prefix nav2_bt_navigator)/share/nav2_bt_navigator/behavior_trees/navigate_to_pose_w_replanning_and_recovery.xml



Pour aller plus loin

Construire des plugins

- https://docs.nav2.org/jazzy/tutorials/plugin_tutorials/
- Poly d'Olivier p185
- exercice bonus : écrire un plugin qui permet d'exclure une zone de la carte, y interdisant la navigation.
- exercice bonus : plugins control et plugins planification (poly Olivier)